7-BIT HAMMING CODE

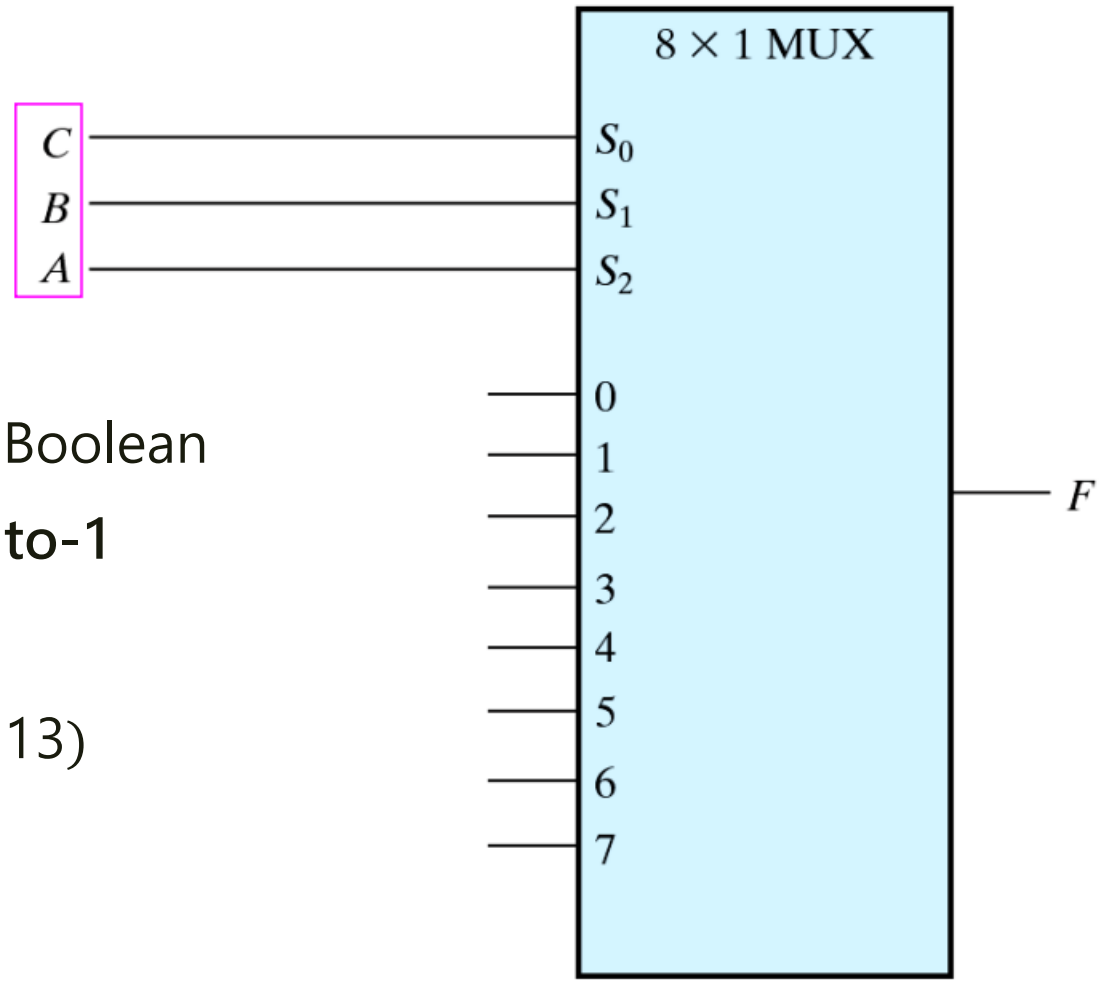
B1130040045 許育菖

2023/11/17

EXERCISE 1

Boolean Function Implementation

題目說明



- Implement and verify the following Boolean function of 4 input variable using **8-to-1 MUX**
- F(A, B, C, D) = $\sum$(1, 2, 5, 8, 9, 10, 12, 13)

Verilog code

分別判斷ABC三位元在各種情況時，F與D之間的關係

```
`timescale 1ns / 1ps

module exercise1(
    output reg F,
    input A, B, C, D);

    always @ (A, B, C, D)
        case(4*A + 2*B + C)
            3'b000:    F = D;
            3'b001:    F = (!D);
            3'b010:    F = D;
            3'b011:    F = 0;
            3'b100:    F = 1;
            3'b101:    F = (!D);
            3'b110:    F = 1;
            3'b111:    F = 0;

        endcase
endmodule
```

F(A, B, C, D)

$= \sum(1, 2, 5, 8, 9, 10, 12, 13)$

A	B	C	D	F
0	0	0	0	
0	0	0	1	1
0	0	1	0	1
0	0	1	1	
0	1	0	0	
0	1	0	1	1
0	1	1	0	
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	
1	1	0	0	1
1	1	0	1	1
1	1	1	0	
1	1	1	1	

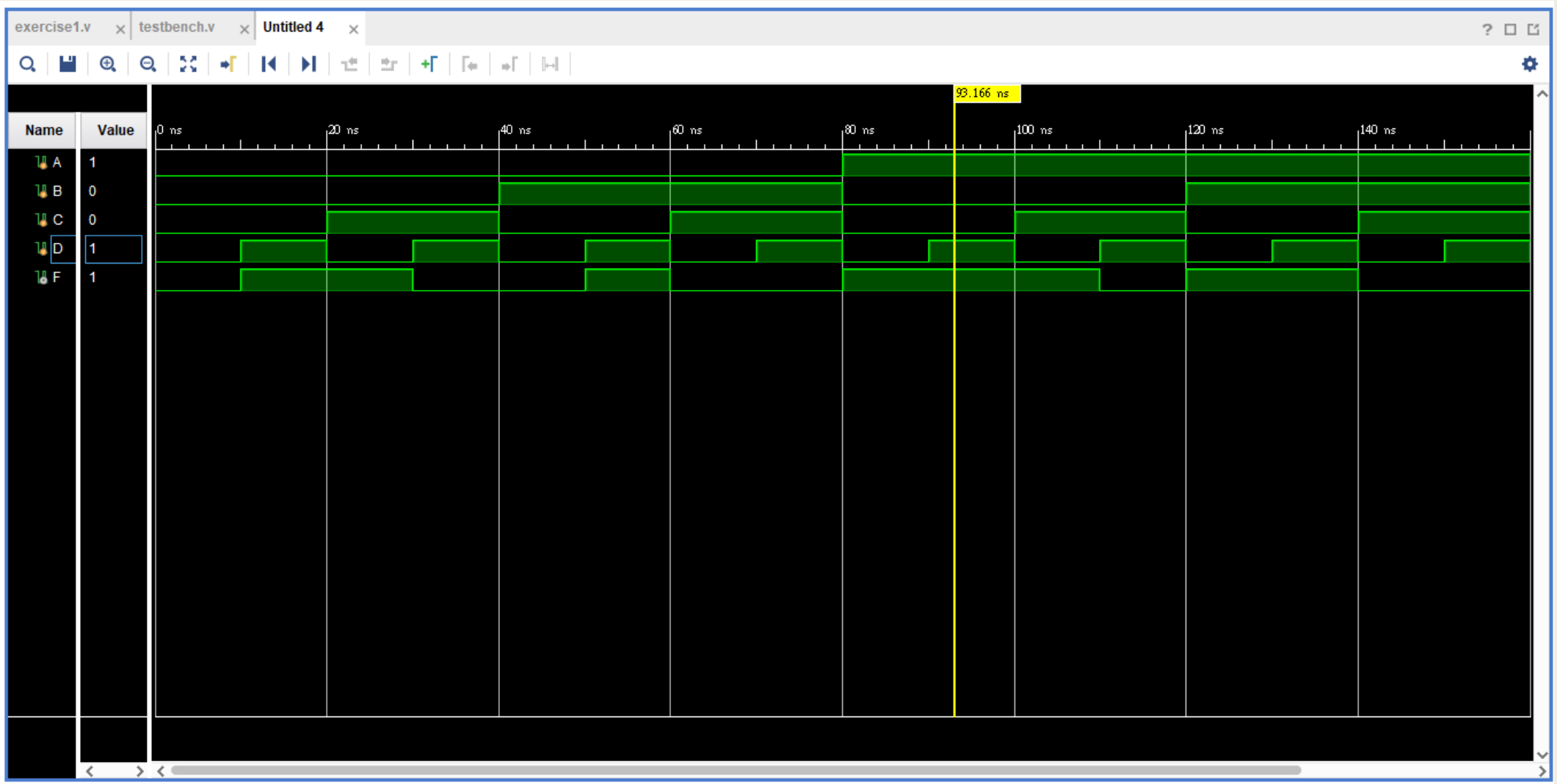
真值表

testbench

輸入二進制的0~15

```
`timescale 1ns / 1ps

module testbench();
    reg A, B, C, D;
    wire F;
exercise1 UO (.A(A), .B(B), .C(C), .D(D), .F(F));
initial begin
    A = 1'b0;   B = 1'b0;   C = 1'b0;    D = 1'b0;
    #10
    A = 1'b0;   B = 1'b0;   C = 1'b0;    D = 1'b1;
    #10
    A = 1'b0;   B = 1'b0;   C = 1'b1;    D = 1'b0;
    #10
    A = 1'b0;   B = 1'b0;   C = 1'b1;    D = 1'b1;
    #10
    A = 1'b0;   B = 1'b1;   C = 1'b0;    D = 1'b0;
    #10
    A = 1'b0;   B = 1'b1;   C = 1'b0;    D = 1'b1;
    #10
    A = 1'b0;   B = 1'b1;   C = 1'b1;    D = 1'b0;
    #10
    A = 1'b0;   B = 1'b1;   C = 1'b1;    D = 1'b1;
    #10

    //

    A = 1'b1;   B = 1'b0;   C = 1'b0;    D = 1'b0;
    #10
    A = 1'b1;   B = 1'b0;   C = 1'b0;    D = 1'b1;
    #10
    A = 1'b1;   B = 1'b0;   C = 1'b1;    D = 1'b0;
    #10
    A = 1'b1;   B = 1'b0;   C = 1'b1;    D = 1'b1;
    #10
    A = 1'b1;   B = 1'b1;   C = 1'b0;    D = 1'b0;
    #10
    A = 1'b1;   B = 1'b1;   C = 1'b0;    D = 1'b1;
    #10
    A = 1'b1;   B = 1'b1;   C = 1'b1;    D = 1'b0;
    #10
    A = 1'b1;   B = 1'b1;   C = 1'b1;    D = 1'b1;
    #10
    $finish;
end
endmodule
```

實驗結果

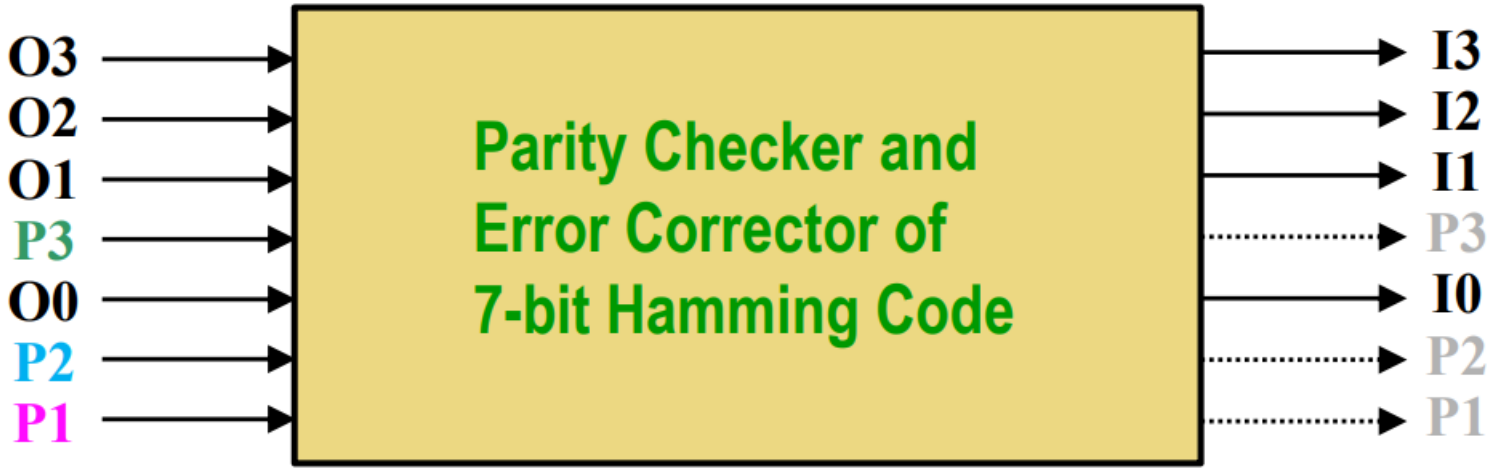

exercise1.v
testbench.v
Untitled 4
93.166 ns
Name
Value
A 1
B 0
C 0
D 1
F 1
0 ns
20 ns
40 ns
60 ns
80 ns
100 ns
120 ns
140 ns

EXERCISE 2

Parity Checker and Error Corrector

題目說明

- Design and verify the Parity Checker and Error Corrector of 7-bit Hamming Code using **Structural level** (Gate-level) **modeling**



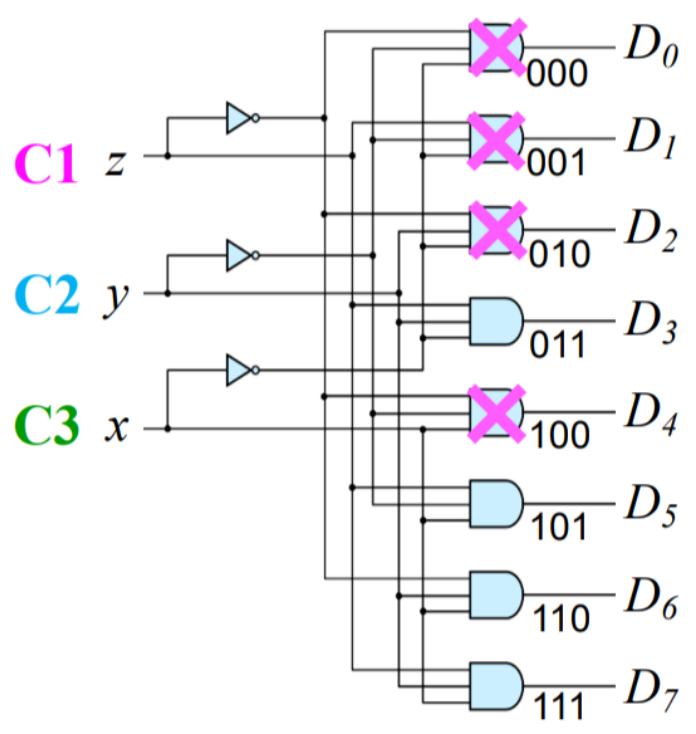
Verilog code

```
`timescale 1ns / 1ps

module exercise2(
    output I3, I2, I1, I0,
    input O3, O2, O1, P3, O0, P2, P1);

    wire C1, C2, C3;

    xor(C3, O3, O2, O1, P3);
    xor(C2, O3, O2, O0, P2);
    xor(C1, O3, O1, O0, P1);

    wire D3, D5, D6, D7;
    and(D3, (!C3), C2, C1);
    and(D5, C3, (!C2), C1);
    and(D6, C3, C2, (!C1));
    and(D7, C3, C2, C1);


    xor(I0, O0, D3);
    xor(I1, O1, D5);
    xor(I2, O2, D6);
    xor(I3, O3, D7);
endmodule
```

$$C3 = O3 \oplus O2 \oplus O1 \oplus P3$$

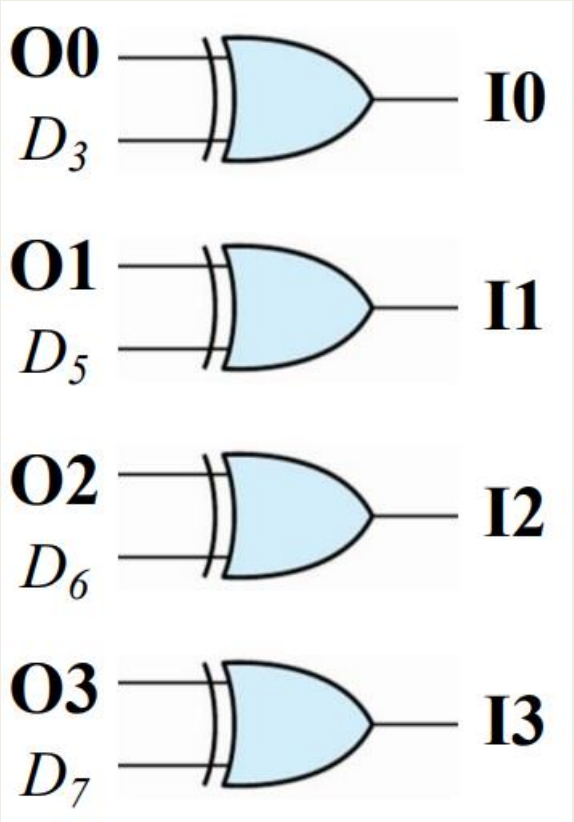
$$C2 = O3 \oplus O2 \oplus O0 \oplus P2$$

$$C1 = O3 \oplus O1 \oplus O0 \oplus P1$$



Verilog code

```
`timescale 1ns / 1ps

module exercise2(
    output I3, I2, I1, I0,
    input O3, O2, O1, P3, O0, P2, P1);

    wire C1, C2, C3;

    xor(C3, O3, O2, O1, P3);
    xor(C2, O3, O2, O0, P2);
    xor(C1, O3, O1, O0, P1);

    wire D3, D5, D6, D7;
    and(D3, (!C3), C2, C1);
    and(D5, C3, (!C2), C1);
    and(D6, C3, C2, (!C1));
    and(D7, C3, C2, C1);


    xor(I0, O0, D3);
    xor(I1, O1, D5);
    xor(I2, O2, D6);
    xor(I3, O3, D7);
endmodule
```



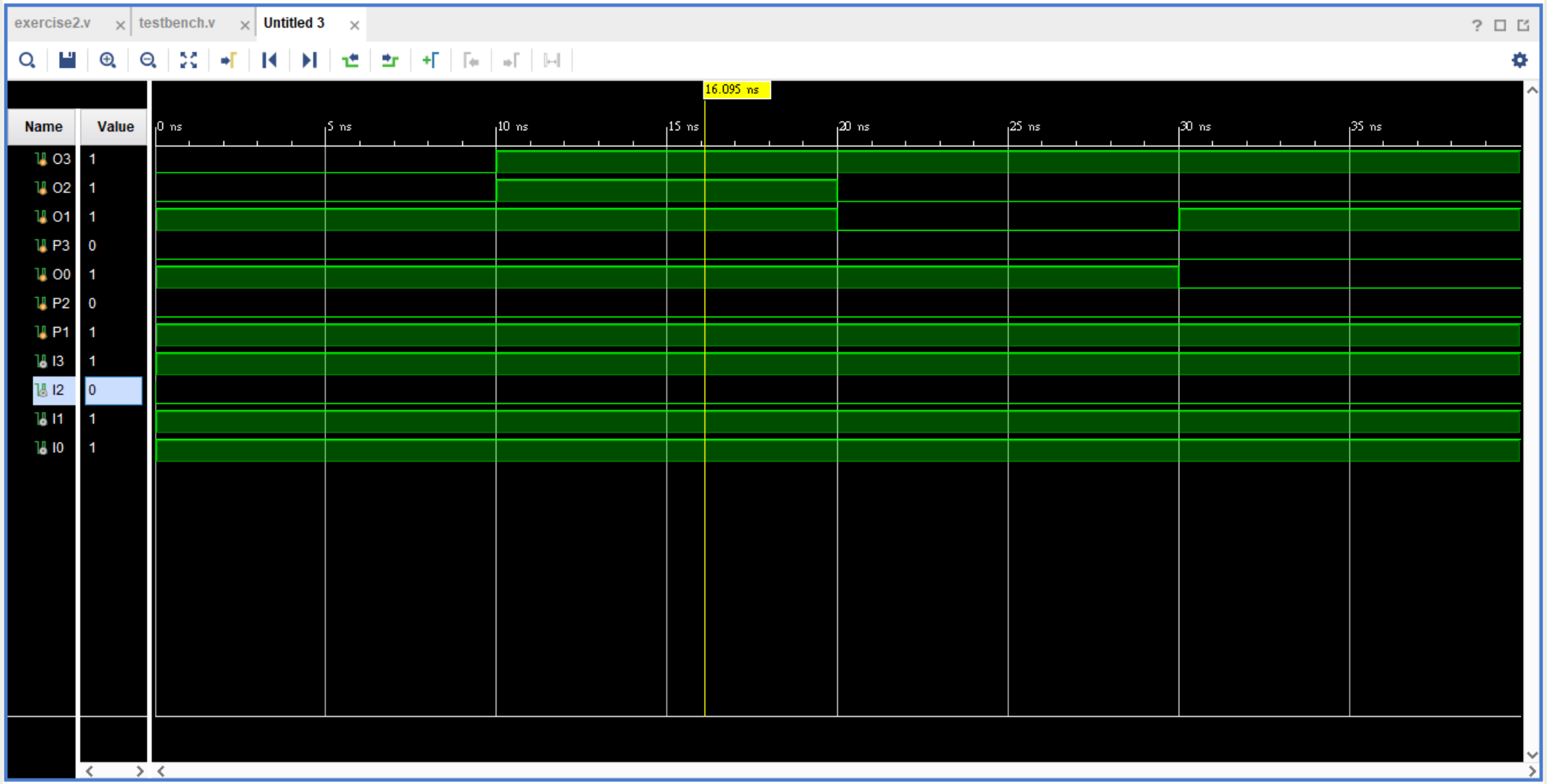
testbench

```
`timescale 1ns / 1ps

module testbench();
    reg O3, O2, O1, P3, O0, P2, P1;
    wire I3, I2, I1, I0;
exercise2
UO(.O3(O3), .O2(O2), .O1(O1), .P3(P3), .O0(O0), .P2(P2), .P1(P1), .I3(I3), .I2(I2), .I1(I1), .I0(I0));
initial begin
    O3 = 1'b0;  O2 = 1'b0;  O1 = 1'b1;  P3 = 1'b0;  O0 = 1'b1; P2 = 1'b0; P1 = 1'b1;
    #10;

    O3 = 1'b1;  O2 = 1'b1;  O1 = 1'b1;  P3 = 1'b0;  O0 = 1'b1; P2 = 1'b0; P1 = 1'b1;
    #10

    O3 = 1'b1;  O2 = 1'b0;  O1 = 1'b0;  P3 = 1'b0;  O0 = 1'b1; P2 = 1'b0; P1 = 1'b1;
    #10

    O3 = 1'b1;  O2 = 1'b0;  O1 = 1'b1;  P3 = 1'b0;  O0 = 1'b0; P2 = 1'b0; P1 = 1'b1;
    #10
    $finish;
end
endmodule
```

Test bench:

0	0	1	0	1	0	1
1	1	1	0	1	0	1
1	0	0	0	1	0	1
1	0	1	0	0	0	1

實驗結果


exercise2.v
testbench.v
Untitled 3
16.095 ns
Name
Value
O3 1
O2 1
O1 1
P3 0
O0 1
P2 0
P1 1
I3 1
I2 0
I1 1
I0 1
0 ns
5 ns
10 ns
15 ns
20 ns
25 ns
30 ns
35 ns

實驗心得

這次實驗實作了 8-to-1 MUX 以及漢明碼的修正器。實驗第二題只要按照PPT上的步驟按部就班就可以順利的完成；然而第一題比較麻煩一點，雖然程式碼看上去比第二題還簡約，但我一開始其實不知道要如何下手，尤其是如何把A、B、C三個一位元的數表示成一個三位元的數字。我在程式碼中所撰寫的case(4*A + 2*B + C)是後來詢問朋友才知道有這個寫法，而朋友也是無意中才發現原來可以這樣寫。這個部份是我認為這次實驗最困難的地方，我覺得有部分原因在於Behavior level modeling是我們才剛接觸到的modeling 。